

Universitatea Transilvania din Braşov

Facultatea de Matematică şi Informatică

Specializarea: Informatică Aplicată

Studiu Preliminar pentru Lucrarea de Licenţă
Impactul Cuantizării Numerice (FP32 / FP16 / FP8)
asupra unui Vision Transformer Antrenat pe
CIFAR-10

Student:

Tudose Alexandru
Grupa 10LF333
An III, Informatică Aplicată

Coordonator:

Conf. dr. ing. Honorius Gâlmeanu
Departamentul de Informatică

Braşov, 2026

Cuprins

1	Introducere și Motivație	2
2	Arhitectura Modelului — Vision Transformer	2
2.1	Principii fundamentale	2
2.2	Configurația utilizată	3
3	Design Experimental	4
3.1	Factori studiați	4
3.2	Pipeline de augmentare	5
3.3	Implementarea preciziei mixte FP16	5
3.4	Implementarea cuantizării FP8 post-training	5
4	Rezultate Experimentale	6
4.1	Comparație cantitativă globală	6
4.2	Evoluția acurateții pe setul de validare	7
4.3	Curbele de loss	9
4.4	Comparații finale — Acuratețe, Overfitting, Timp	10
4.5	Impactul cuantizării FP8 post-training	13
4.6	Profil hardware și consum energetic	14
5	Analiză și Discuții	15
5.1	Efectul preciziei numerice — FP32 vs. FP16	15
5.2	Efectul augmentării și regularizării	16
5.3	Viabilitatea FP8 pentru deployment	16
5.4	Limitări ale studiului	16
6	Concluzii și Direcții Viitoare	17
6.1	Concluzii	17
6.2	Pasul următor — Obiectivul principal al lucrării de licență	17

1 Introducere și Motivație

Modelele de tip *Large Language Model* și modelele generative de imagini (e.g. familia FLUX de la Black Forest Labs [4]) au atins performanțe remarcabile, dar vin cu un cost computațional considerabil — atât la antrenare, cât și la inferență. Rularea acestor modele pe dispozitive cu resurse limitate (laptop, telefon mobil) fără a sacrifica calitatea este o problemă activă de cercetare.

Cuantizarea numerică — reducerea preciziei cu care sunt stocate și procesate ponderile și activările unui model neural — este una dintre cele mai eficiente tehnici de compresie a modelelor. Standardul din industrie a evoluat de la FP32 (precizie completă) la FP16/BF16 (precizie mixtă, adoptată pe scară largă), și mai recent la FP8, suportat nativ pe acceleratoarele NVIDIA H100/H200 și implementat nativ în PyTorch 2.1+.

Scopul acestui studiu

Prezentul document raportează un **studiu preliminar** cu rolul de a înțelege practic comportamentul cuantizării înainte de aplicarea sa pe modele generative complexe. Folosind arhitectura `vit_tiny_patch16_224` antrenată pe CIFAR-10, am investigat:

1. **Efectul preciziei** — cât de multă acuratețe se pierde la $FP32 \rightarrow FP16 \rightarrow FP8$?
2. **Efectul regularizării** — poate augmentarea extinsă a datelor să compenseze un eventual deficit de precizie?
3. **Viabilitatea FP8 PTQ** — este cuantizarea post-training la FP8 viabilă pentru deployment fără re-antrenare?
4. **Profilul hardware** — care este impactul preciziei asupra consumului energetic și utilizării resurselor pe Apple Silicon?

Rezultatele acestui studiu vor informa **etapa principală a lucrării de licență**: cuantizarea modelului generativ FLUX.1 [schnell] pe acceleratoare NVIDIA cu suport nativ FP8 (Google Colab / Kaggle).

2 Arhitectura Modelului — Vision Transformer

2.1 Principii fundamentale

Vision Transformer (ViT) [1] aplică mecanismul *self-attention* din arhitectura Transformer direct pe imagini. O imagine de dimensiune $H \times W \times C$ este împărțită în

$N = HW/P^2$ patch-uri de dimensiune $P \times P$, fiecare aplatizat și proiectat liniar într-un spațiu de dimensiune D .

Blocul central Transformer operează cu *Multi-Head Self-Attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

unde $Q, K, V \in \mathbb{R}^{N \times d_k}$ sunt proiecțiile query, key și value. Fiecare bloc include și o rețea feed-forward:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (2)$$

Ambele sub-straturi sunt înconjurate de conexiuni reziduale și normalizare LayerNorm.

2.2 Configurația utilizată

Varianta **ViT-Tiny** (`vit_tiny_patch16_224`), instantiată prin biblioteca `timm`, are **5.7 milioane de parametri** — dimensiune potrivită pentru experimentare locală pe hardware limitat.

Tabela 1: Parametrii arhitecturali ai ViT-Tiny utilizat în experimente

Proprietate	Valoare
Parametri totali	5.7 milioane
Patch size	16×16 pixeli
Input resolution	224×224 (redimensionat de la 32×32)
Embedding dim	192
Număr blocuri	12
Număr capete (MHA)	3

Notă privind redimensionarea: Imaginile CIFAR-10 au rezoluția nativă de 32×32 pixeli. Acestea au fost redimensionate prin interpolare bicubică la 224×224 pixeli, rezoluția cerută de ViT. Deși această redimensionare introduce artefacte vizuale, CIFAR-10 rămâne un dataset de referință util pentru compararea configurațiilor de antrenare în condiții controlate.

3 Design Experimental

3.1 Factori studiați

Experimentele urmează un **design factorial** 2×2 cu doi factori indepedeni, plus un experiment adițional de cuantizare post-training:

- **Factorul 1 — Precizie numerică:** FP32 vs. FP16 cu Automatic Mixed Precision (AMP)
- **Factorul 2 — Strategie de augmentare:** Basic vs. Extended (însoțită de regularizare completă)
- **Adițional:** Cuantizare FP8 post-training (PTQ) aplicată pe cel mai bun check-point

Tabela 2: Cele 5 configurații experimentale

Experiment	Precizie	Aug.	Regularizare	Scop
BaseFP32	FP32	Basic	WD=0.05	Referință
BaseFP16	FP16 (AMP)	Basic	WD=0.05	Efect precizie
AugmFP32	FP32	Extended	WD=0.1 + LS + GC + warmup	Efect augmentare
AugmFP16	FP16 (AMP)	Extended	WD=0.1 + LS + GC + warmup	Configurație optimă
FP8Test	FP8 PTQ	—	—	Viabilitate deployment

WD = weight decay, LS = label smoothing (0.1), GC = gradient clipping (1.0), warmup = 5 epoci

Tabela 3: Hiperparametri comuni tuturor antrenamentelor

Parametru	Valoare
Model	vit_tiny_patch16_224 (5.7M params)
Dataset	CIFAR-10 (50.000 train / 10.000 test, 10 clase)
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	10^{-3} inițial, Cosine Annealing
Loss	CrossEntropyLoss
Batch size	128
Epoci	50
Hardware	MacBook Air, Apple M4 (10-core CPU, 8-core GPU), 16 GB RAM
Backend PyTorch	MPS (Metal Performance Shaders)

3.2 Pipeline de augmentare

Augmentare Basic (configurații Baseline):

- `Resize(224) + RandomCrop(224, padding=28)`
- `RandomHorizontalFlip()`
- Normalizare CIFAR-10: $\mu = [0.491, 0.482, 0.447]$, $\sigma = [0.247, 0.244, 0.262]$

Augmentare Extended (adăugat față de Basic):

- `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)`
- `RandomRotation(15°)`
- `RandomErasing(p=0.5, scale=(0.02, 0.33))` [7]

3.3 Implementarea preciziei mixte FP16

Configurațiile FP16 folosesc `torch.amp.autocast` cu `GradScaler` pentru antrenare în precizie mixtă. Operațiile critice (softmax, acumulare gradienti) rămân în FP32, în timp ce matmul-urile și convoluțiile sunt executate în FP16. Pe arhitectura Apple M4, backend-ul MPS nu dispune de unități Tensor Core specializate pentru FP16 (prezente pe GPU NVIDIA H100/A100), ceea ce influențează raportul de viteză, discutat în Secțiunea 5.1.

3.4 Implementarea cuantizării FP8 post-training

Cuantizarea FP8 a fost aplicată *post-training* pe checkpoint-ul AugmFP16 (cel mai bun model), folosind formatul `float8_e4m3fn` (4 biți exponent, 3 biți mantisă) disponibil nativ în PyTorch 2.1+ [3]. Au fost evaluate două strategii:

- **Per-tensor:** un singur factor de scalare calculat din $\max(|W|)$ al întregului tensor de ponderi
- **Per-channel:** factori de scalare individuali per canal, calculați din distribuția reală a fiecărui set de ponderi

Notă tehnică importantă: O implementare inițială a folosit range-ul teoretic maxim al formatului FP8 (± 240) în loc de range-ul real al ponderilor ($\approx \pm 0.64$). Aceasta a condus la zeroing complet al tuturor parametrilor (acuratețe 10%, echivalentă cu o predicție aleatoare pentru 10 clase). Eroarea a subliniat că **calibrarea factorilor de scalare la distribuția reală a datelor este esențială** — range-ul teoretic al formatului numeric nu este relevant pentru scalarea ponderilor unui model antrenat.

4 Rezultate Experimentale

4.1 Comparație cantitativă globală

Tabelul 4 centralizează toate metricile relevante pentru cele 4 experimente de antrenare.

Tabela 4: Rezultate complete ale celor 4 experimente

Metrică	BaseFP32	BaseFP16	AugmFP32	AugmFP16
<i>Performanță model</i>				
Best Val Accuracy	78.99%	79.19%	82.88%	83.41%
Final Val Accuracy	78.92%	79.02%	82.71%	83.41%
Final Train Accuracy	97.29%	97.43%	86.80%	85.67%
Overfitting Gap	18.4 pp	18.4 pp	4.1 pp	2.3 pp
Final Val Loss	0.9138	0.9056	0.8999	0.8873
Final Train Loss	0.0834	0.0777	0.7941	0.8214
Convergence Epoch	43	35	45	48
<i>Timp de antrenare</i>				
Timp total	7.26 h	7.45 h	7.38 h	7.09 h
Timp/epocă (med.)	522.9 s	536.2 s	531.3 s	510.3 s
Timp/epocă (std.)	±16.5 s	±22.0 s	±27.7 s	±18.0 s
<i>Utilizare hardware</i>				
CPU Avg	2.6%	2.6%	5.2%	4.8%
CPU Max	54.3%	58.9%	59.1%	62.6%
RAM Avg	83.6%	81.6%	86.0%	81.6%
RAM Max	89.3%	86.4%	90.8%	86.8%
Thermal throttling	Nu	Nu	Nu	Nu

4.2 Evoluția acurateții pe setul de validare

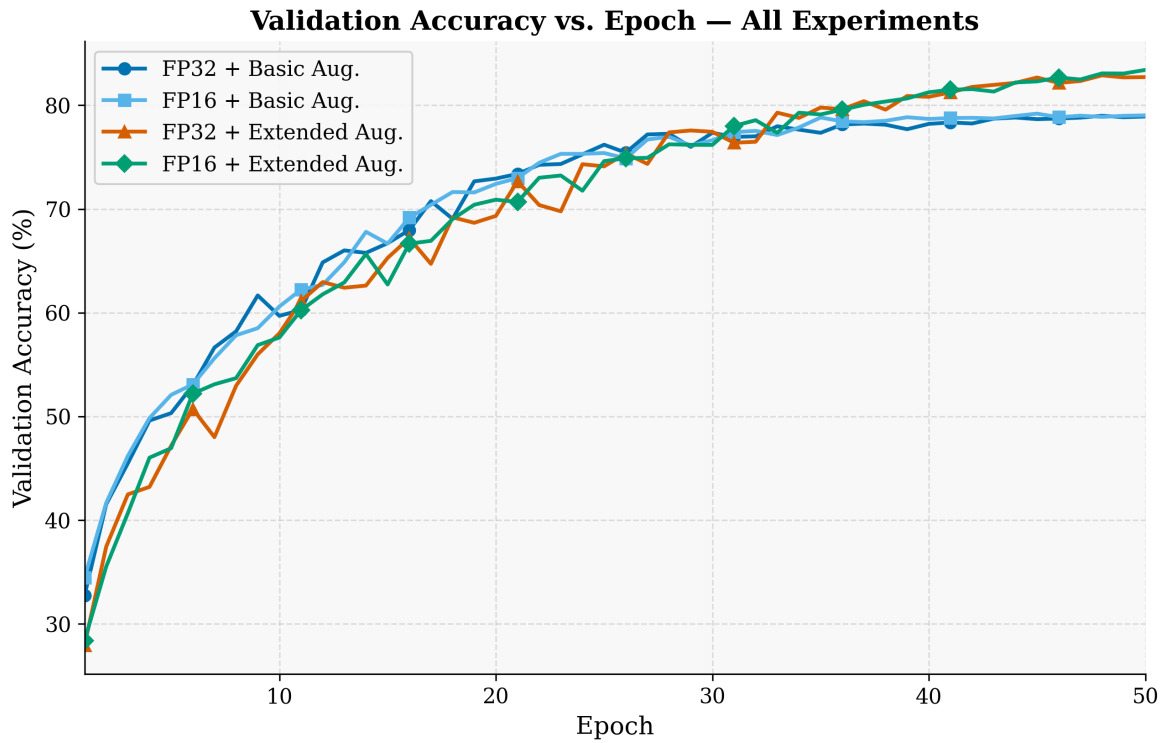


Figura 1: Acuratețea de validare pentru toate 4 experimente suprapuse, pe parcursul celor 50 de epoci. În primele ~30 de epoci toate configurațiile evoluează similar, cu curbe inter-amestecate. Separarea netă între Augmented (portocaliu, verde) și Baseline (albastru închis și deschis) devine evidentă abia după epoca 30: modelele Baseline intră într-un platou, în timp ce modelele Augmented continuă să crească până la finalul antrenării.

Train vs. Validation Accuracy — Overfitting Visualisation

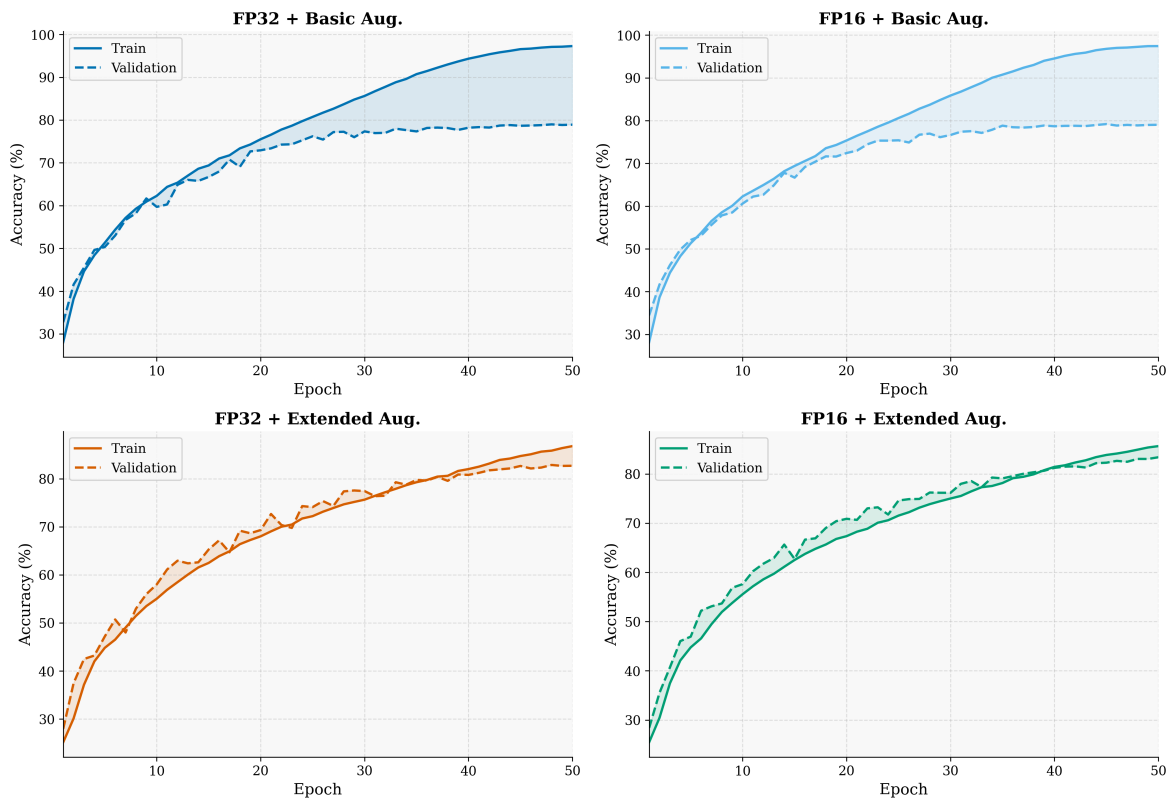


Figura 2: Train vs. Validation Accuracy pentru fiecare configurație (grid 2×2). Zona umbrită reprezintă gap-ul de generalizare. **Rândul de sus** (Baseline): gap-ul crește continuu pe tot parcursul antrenării, atingând 18.4 pp la finalul celor 50 de epoci. Train accuracy ajunge la $\sim 97\%$ în timp ce val accuracy stagnează la $\sim 79\%$. **Rândul de jos** (Augmented): gap-ul este mic și relativ stabil. Cele două curbe (train și val) rămân apropiate pe tot parcursul antrenării, indicând o generalizare sănătoasă. Remarcabil este faptul că în primele epoci val accuracy depășește train accuracy — efect al augmentărilor care fac antrenarea mai dificilă decât evaluarea.

4.3 Curbele de loss

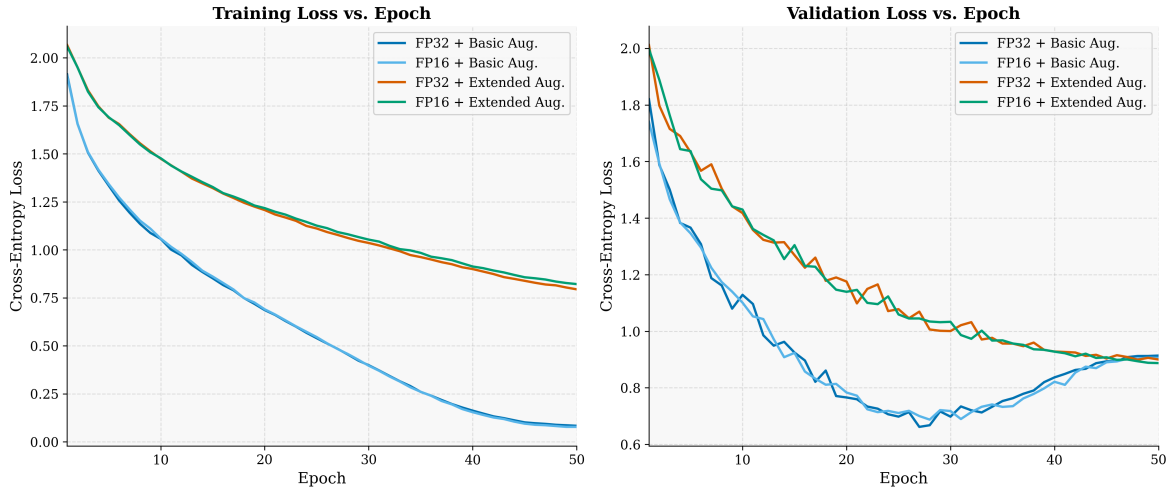


Figura 3: Stânga: Training loss vs. epocă. Baseline-urile (albastru închis și deschis) scad agresiv spre valori mici (≈ 0.08), indicând memorarea datelor de antrenare. Modelele Augmented mențin un training loss ridicat (≈ 0.80) datorită dificultății augmentărilor combinate cu label smoothing, care penalizează probabilitățile de predicție prea mari. **Dreapta:** Validation loss vs. epocă. Baseline-ul FP32 (albastru închis) prezintă clar **forma în U**: atinge minimumul în jurul epocii 20–25, după care crește progresiv, semn că modelul a început să memoreze zgomotul din datele de antrenare. Modelele Augmented continuă să scadă monoton până la finalul celor 50 de epoci, fără nicio inversare a tendinței.

Observația esențială din Figura 3 este că **un training loss mic nu implică un model mai bun**: BaseFP32 are cel mai mic training loss (≈ 0.08), dar cel mai mare validation loss (0.9138) și cea mai mică acuratețe pe test (78.99%). AugmFP16 are training loss ridicat (≈ 0.82), dar cel mai bun val accuracy (83.41%).

4.4 Comparații finale — Acuratețe, Overfitting, Timp

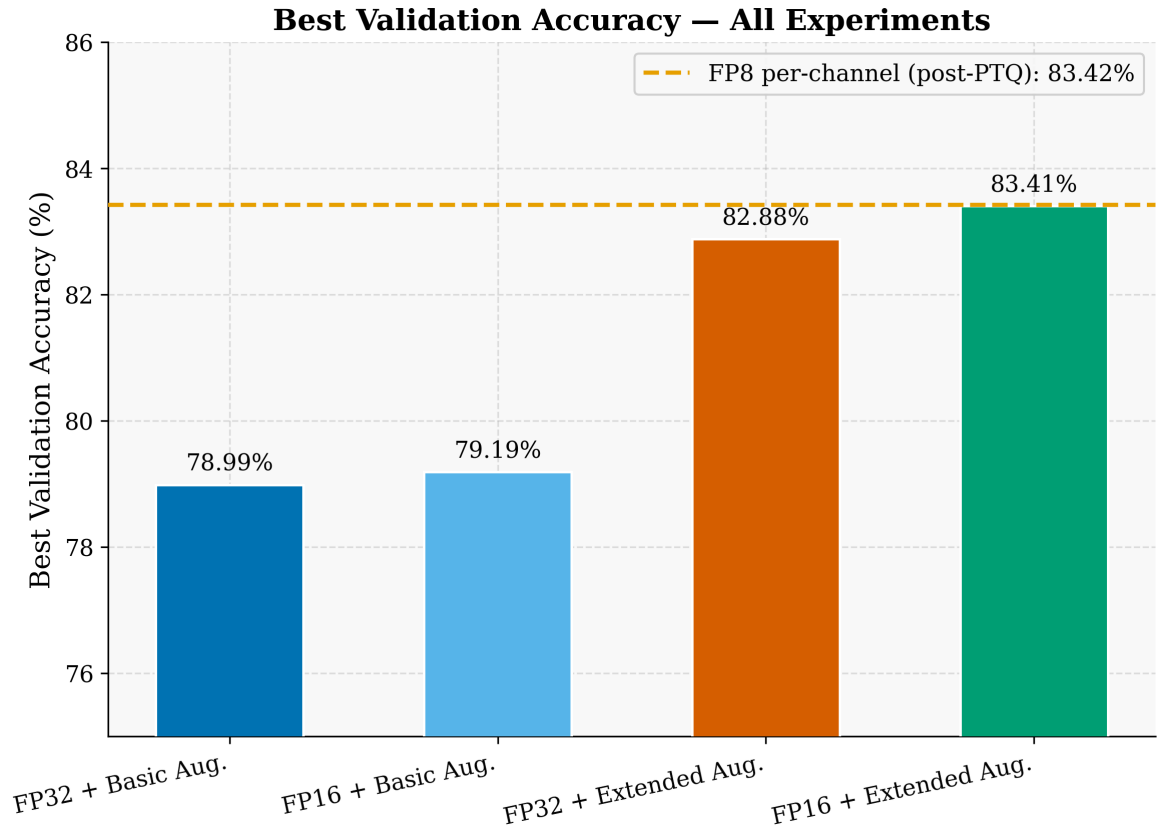


Figura 4: Acuratețea maximă pe setul de validare pentru fiecare configurație. Linia portocalie punctată reprezintă acuratețea modelului cuantizat FP8 per-channel (83.42%), aplicat post-training pe checkpoint-ul AugmFP16. Faptul că linia FP8 se situează cu 0.01 pp deasupra bar-ului AugmFP16 (83.41%) reflectă variabilitatea minimă naturală a evaluării, nu un câștig real de acuratețe prin cuantizare.

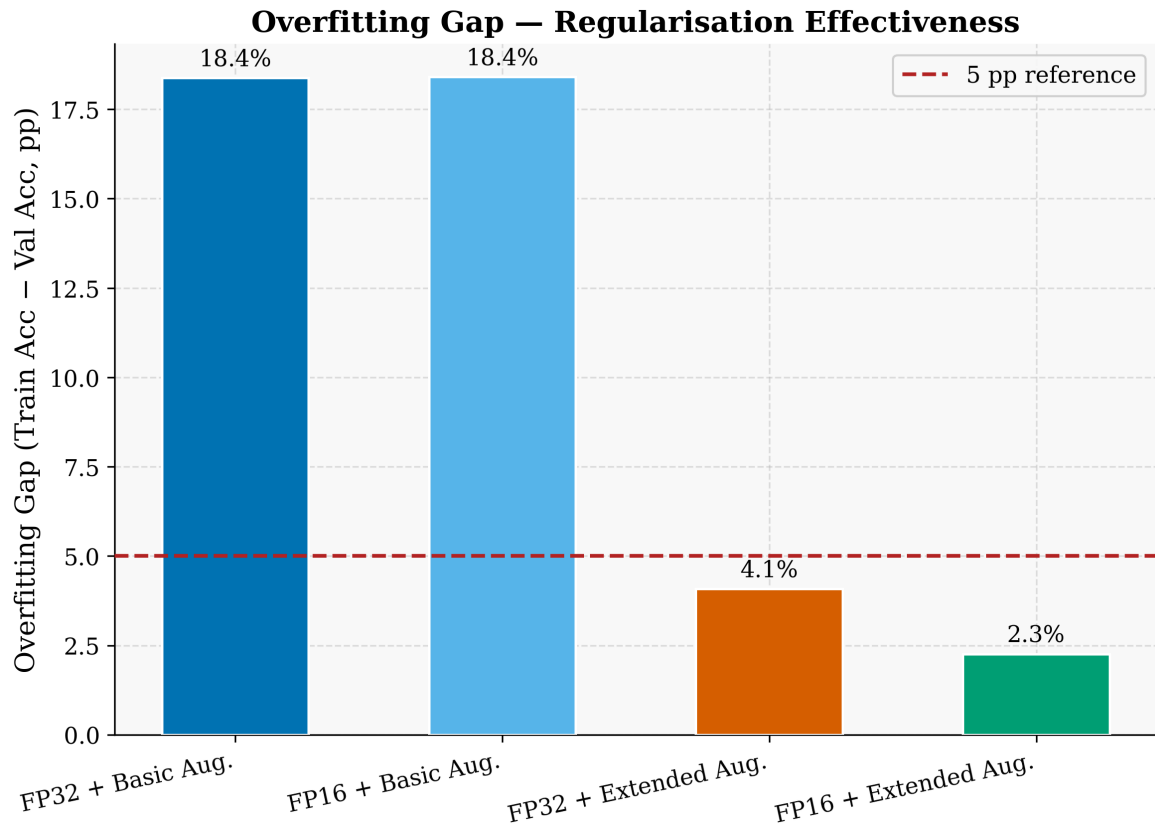


Figura 5: Gap-ul de overfitting (Train Acc — Val Acc) la finalul antrenării. Linia roșie punctată la 5 pp reprezintă un prag practic de referință pentru generalizare acceptabilă. Baseline-urile se situează la 18.4 pp — de peste trei ori pragul — indiferent de precizia utilizată. AugmFP32 și AugmFP16 coboară la 4.1 pp respectiv 2.3 pp, ambele sub prag.

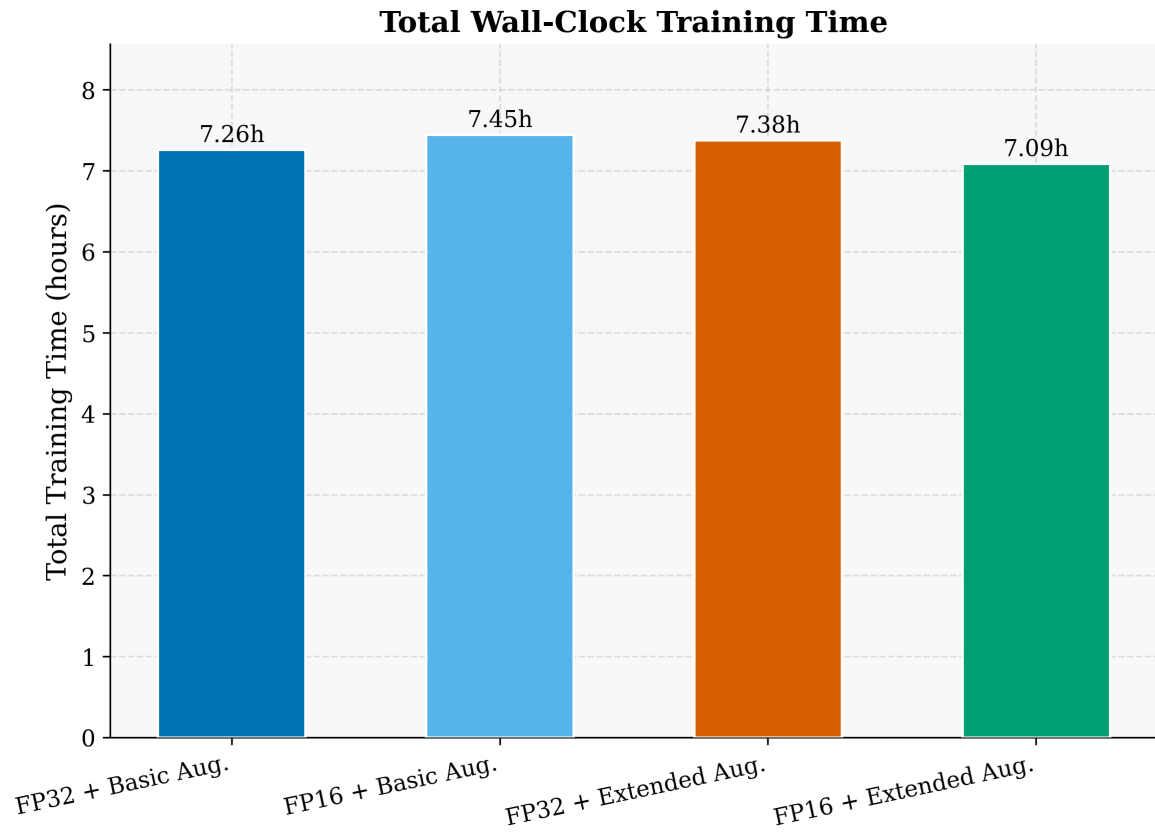


Figura 6: Timpul total de antrenare (50 epoci) pentru fiecare configurație. Diferențele sunt minore: toate experimentele se încadrează în intervalul 7.09–7.45 ore. Contrar așteptărilor bazate pe GPU NVIDIA, FP16 nu aduce un avantaj de viteză pe Apple M4 (MPS) — AugmFP16 este chiar puțin mai rapid (7.09 h), dar aceasta se datorează probabil variației de la o rulare la alta, nu preciziei numerice.

4.5 Impactul cuantizării FP8 post-training

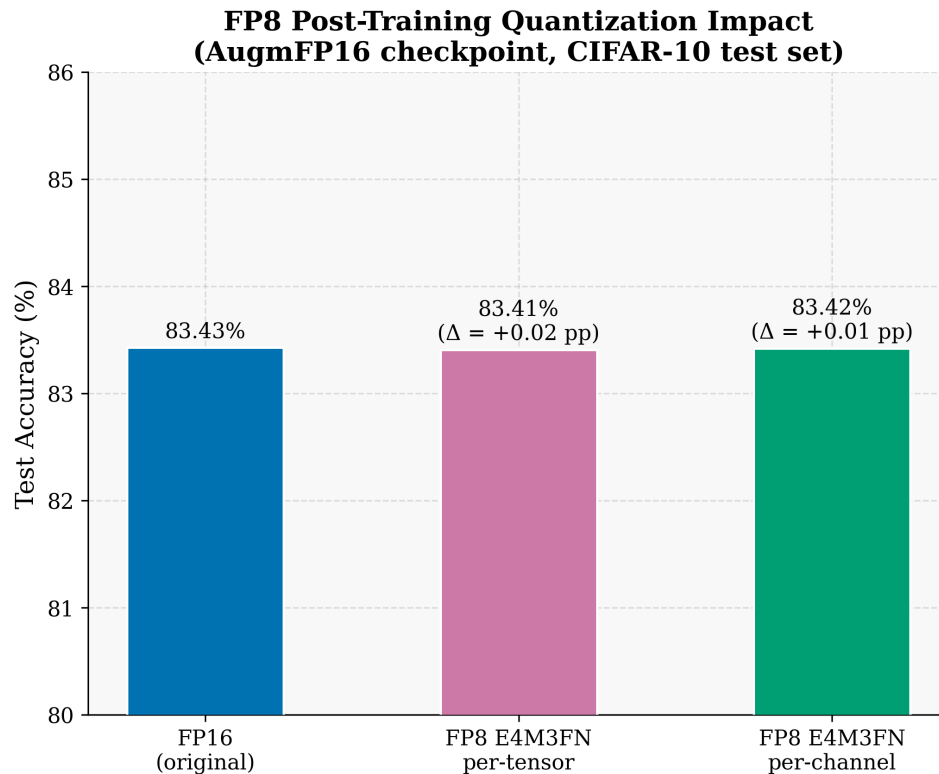


Figura 7: Impactul cuantizării FP8 E4M3FN asupra acurateții pe setul de test. Modelul original (FP16) obține 83.43%. Cuantizarea per-tensor produce 83.41% ($\Delta = 0.02$ pp), iar cuantizarea per-channel produce 83.42% ($\Delta = 0.01$ pp). Ambele degradează cu mai puțin de 0.03 pp — practic neglijabil la scara variabilității naturale a evaluării.

Tabela 5: Rezultatele cuantizării FP8 post-training aplicate pe AugmFP16

Model	Acuratețe	Degradare	Format	Strategie
Original (AugmFP16)	83.43%	—	float16	—
Cuantizat	83.41%	0.02 pp	float8_e4m3fn	Per-tensor
Cuantizat	83.42%	0.01 pp	float8_e4m3fn	Per-channel

Cuantizarea per-channel produce rezultate marginale superioare față de per-tensor, deoarece fiecare canal are propriul factor de scalare adaptat la distribuția sa de ponderi, reducând eroarea de rotunjire față de un factor global per tensor.

4.6 Profil hardware și consum energetic

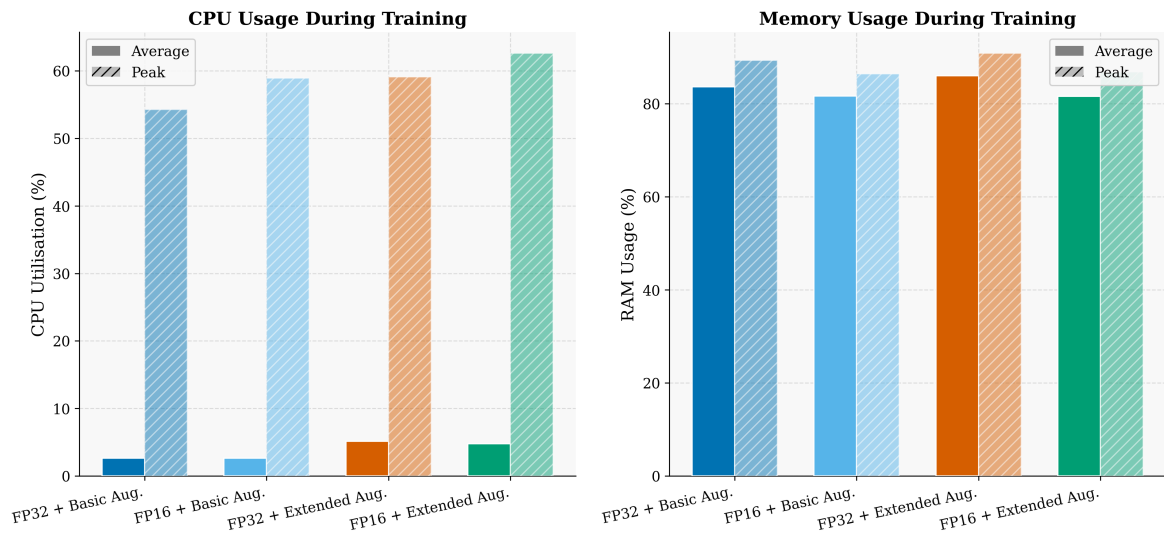


Figura 8: Utilizarea medie și de vârf a CPU-ului (stânga) și a memoriei RAM (dreapta) în timpul antrenării. Utilizarea medie a CPU este foarte redusă (2.6–5.2%), confirmând că GPU-ul MPS preia covârșitor calculele de matmul și backpropagation. Vârfurile de CPU (54–63%) apar la operațiile de preprocesare a imaginilor (augmentările rulează pe CPU) și la checkpointing. Memoria RAM este utilizată intens (81–87%), modelul și batch-urile rezidând permanent în memoria unificată Apple M4.

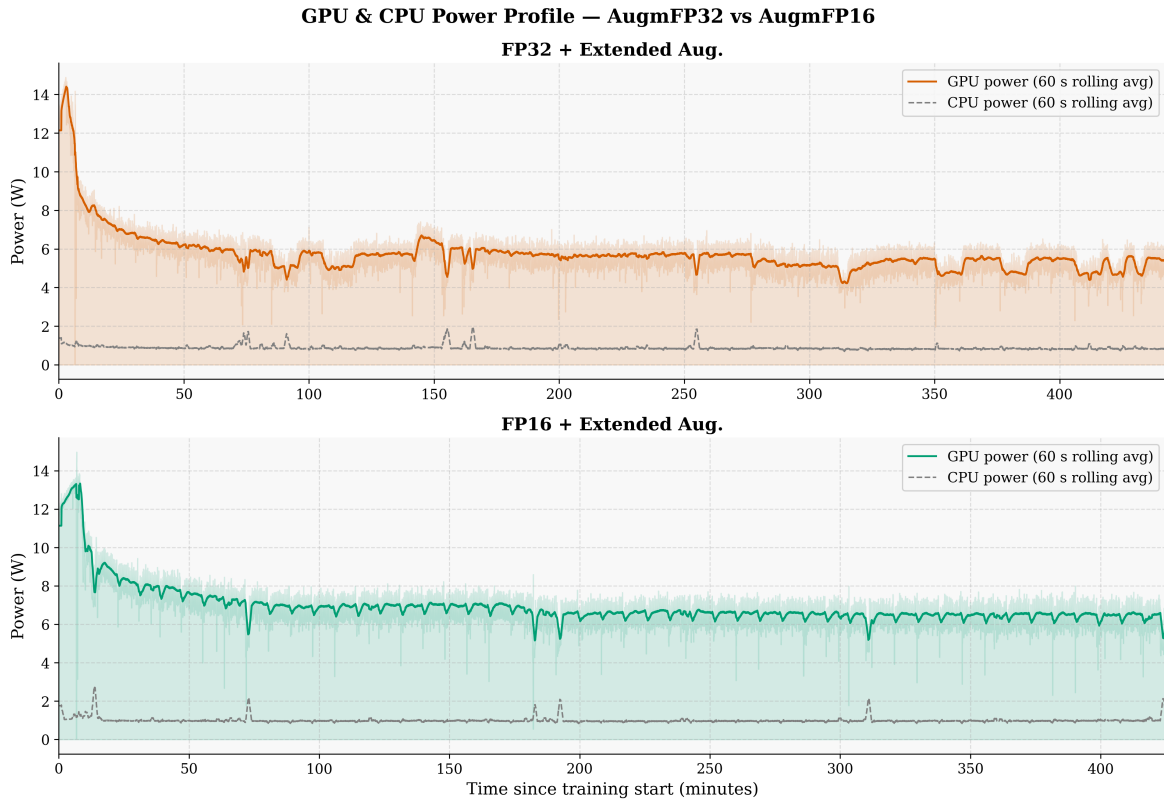


Figura 9: Profilul de putere GPU (medie mobilă de 60 s) pentru AugmFP32 (sus, portocaliu) și AugmFP16 (jos, verde) pe durata întregii antrenări (≈ 430 minute). Ambele configurații prezintă un vârf inițial la pornire ($\approx 13\text{--}14$ W) datorat inițializării MPS. În regim stabil: **AugmFP32** oscilează în jurul valorii de ≈ 5 W cu variabilitate mai mare; **AugmFP16** se stabilizează la ≈ 6 W cu variabilitate mai redusă și consum mai constant. Faptul că FP16 consumă mai multă putere GPU este contraintuitiv, dar se explică prin faptul că Mixed Precision menține GPU-ul activ mai continuu (throughput mai mare), reducând perioadele de idle față de FP32.

5 Analiză și Discuții

5.1 Efectul preciziei numerice — FP32 vs. FP16

Comparând **BaseFP32** cu **BaseFP16** (factorul preciziei în absența augmentărilor), diferența de acuratețe este de doar **+0.21 pp** în favoarea FP16 (79.19% vs. 78.99%). Aceasta se încadrează în variabilitatea normală a antrenamentelor stochastice și nu poate fi atribuită preciziei numerice per se.

Contrar comportamentului bine-documentat pe GPU-uri NVIDIA — unde AMP aduce speedup-uri de $1.5\text{--}3\times$ datorită unităților Tensor Core specializate pentru FP16 — pe Apple M4 (MPS) antrenarea FP16 este cu **0.19 ore mai lentă** decât FP32 (7.45 h vs. 7.26 h). Overhead-ul conversiei dinamice a tensorilor în AMP anulează potentialul avantaj de bandwidth pe această arhitectură.

Concluzie: Pe MPS, trecerea la FP16 este neutră ca viteză și nu costă nimic în acuratețe. Rămâne relevantă ca pregătire pentru deployment pe hardware NVIDIA unde avantajul de viteză este substanțial.

5.2 Efectul augmentării și regularizării

Acesta este **cel mai impactant factor** din întregul experiment. Trecerea de la augmentare Basic la Extended (cu label smoothing, gradient clipping și LR warmup) produce:

- **FP32:** +3.90 pp acuratețe (78.99% → 82.88%)
- **FP16:** +4.21 pp acuratețe (79.19% → 83.41%)
- Reducerea gap-ului de overfitting de la **18.4 pp** la **4.1 / 2.3 pp**
- Eliminarea formei în U din curba de validation loss

Label smoothing previne modelul să fie prea sigur pe predicțiile sale (penalizează probabilități de predicție prea aproape de 1.0), gradient clipping stabilizează antrenarea în primele epoci, iar augmentările extinse prezintă variații diverse ale aceleași imagini, reducând posibilitatea de memorare a detaliilor specifice.

5.3 Viabilitatea FP8 pentru deployment

Rezultatele din Secțiunea 4.5 sunt remarcabile: o degradare de **0.01–0.02 pp** față de modelul FP16 original, fără nicio re-antrenare. Aceasta confirmă că:

1. Ponderile unui ViT antrenat cu augmentare și regularizare au o distribuție suficient de “bine condiționată” pentru a fi reprezentate cu 8 biți fără pierdere semnificativă.
2. Calibrarea factorilor de scalare la distribuția reală a ponderilor este critică — folosirea range-ului teoretic maxim (± 240) a produs un model nefuncțional (acuratețe 10%).
3. FP8 E4M3FN reduce memoria necesară la inferență cu $\approx 50\%$ față de FP16 și cu $\approx 75\%$ față de FP32, fără niciun cost practic de acuratețe.

5.4 Limitări ale studiului

- **Dataset limitat:** CIFAR-10 este o sarcină de clasificare simplă. Concluziile despre FP8 nu se generalizează automat la generarea de imagini sau detecția de obiecte.

- **Hardware specific:** Toate antrenamentele au rulat pe Apple M4 (MPS). Raportul de viteză FP32/FP16 este fundamental diferit pe NVIDIA CUDA.
- **Model mic:** ViT-Tiny (5.7M params) este cu câteva ordine de mărime sub modelele de producție (e.g. FLUX.1 schnell $\approx$ 12B params). Comportamentul cuantizării poate diferi la scară mai mare.
- **PTQ only:** Nu a fost evaluat Quantization-Aware Training (QAT), care ar putea menține acuratețea mai bine la rate de compresie mai mari.

6 Concluzii și Direcții Viitoare

6.1 Concluzii

Studiul preliminar a răspuns clar la toate întrebările de cercetare propuse:

1. **FP32 $\rightarrow$ FP16:** Fără cost de acuratețe (+0.21 pp); fără beneficiu de viteză pe MPS Apple Silicon (spre deosebire de GPU NVIDIA).
2. **FP16 $\rightarrow$ FP8 PTQ:** Degradare neglijabilă (0.01–0.02 pp), demonstrând că FP8 este viabil pentru deployment fără re-antrenare, cu condiția calibrării corecte a factorilor de scalare.
3. **Augmentarea domină precizia** ca factor de generalizare: +4 pp acuratețe și reducerea overfitting gap-ului de la 18% la 2–4%, indiferent de precizia numerică.
4. **Apple M4** gestionează antrenarea fără thermal throttling, cu consum stabil de $\approx$ 5–6 W GPU pe durate de 7+ ore.

6.2 Pasul următor — Obiectivul principal al lucrării de licență

Pe baza cunoștințelor acumulate despre comportamentul practic al cuantizării, etapa următoare constă în aplicarea FP8 pe un model generativ de dimensiune reală: **FLUX.1 [schnell]** [4] (Black Forest Labs, arhitectură Diffusion Transformer bazată pe ViT, $\approx$ 12B parametri).

Experimentele vor fi rulate pe GPU NVIDIA (Google Colab Pro / Kaggle cu A100 sau H100), unde FP8 este suportat nativ în hardware și oferă speedup-uri reale. Metricile de evaluare planificate:

- **Calitatea imaginilor generate:** FID score, CLIP Score, evaluare vizuală comparativă
- **Viteză de inferență:** latență per imagine (ms/step)

- **Consum de memorie:** VRAM necesar în FP32 vs. FP16 vs. FP8
- **Comparație de cuantizare:** FP16 baseline vs. FP8 per-tensor vs. FP8 per-channel

Bibliografie

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” în *International Conference on Learning Representations (ICLR)*, 2021. [arXiv:2010.11929](#).
- [2] H. Touvron, M. Cord, M. Douze et al., “Training data-efficient image transformers & distillation through attention,” în *International Conference on Machine Learning (ICML)*, 2021. [arXiv:2012.12877](#).
- [3] P. Micikevicius, D. Stosic, N. Burgess et al., “FP8 Formats for Deep Learning,” 2022. [arXiv:2209.05433](#).
- [4] Black Forest Labs, “FLUX.1: A family of flow matching text-to-image models,” *Technical Report*, 2024. [bfl.ai](#).
- [5] PyTorch Team, “Automatic Mixed Precision package — `torch.amp`,” Documentație oficială PyTorch, 2024. [pytorch.org/docs/stable/amp.html](#).
- [6] I. Loshchilov și F. Hutter, “Decoupled Weight Decay Regularization,” în *ICLR*, 2019. [arXiv:1711.05101](#).
- [7] Z. Zhong, L. Zheng, G. Kang et al., “Random Erasing Data Augmentation,” în *AAAI Conference on Artificial Intelligence*, 2020. [arXiv:1708.04896](#).